Home Assignment – Kubernetes, DevOps & FinOps

1. Requirement Understanding

The objective of this assignment is to design, containerize, and deploy a multi-tier architecture on Kubernetes involving one microservice and one database.

Key requirements identified are as follows:

- Expose the Service API Tier externally using Ingress.
- Service API should run with 4 replicas (pods).
- Database tier should run with 1 pod only.
- Database pod should have persistent storage, and data should not be lost if the pod is redeployed.
- Service API should support rolling updates.
- Configuration should be externalized using ConfigMaps.
- Sensitive information should be managed using Kubernetes Secrets.
- The solution should demonstrate scalability through Horizontal Pod Autoscaling (HPA).
- The solution should demonstrate self-healing capabilities.
- The solution should incorporate FinOps principles by optimizing resource utilization, infrastructure sizing, storage allocation, and scalability to achieve a cost-efficient deployment.

---

2. Assumptions

The following assumptions were considered during the design and implementation of the solution:

- A Kubernetes cluster with 2 worker nodes was created assuming it to be sufficient to host the API and database workloads.
- The API workload is expected to operate within the configured resource allocation of 100m CPU and 128Mi memory requests, with limits of 500m CPU and 512Mi memory.
- CPU utilization is considered an appropriate metric for workload scaling and is therefore used by the Horizontal Pod Autoscaler (HPA).
- The HPA is configured with a minimum of 4 replicas and a maximum of 8 replicas to balance application availability and infrastructure cost.

- The target CPU utilization threshold for autoscaling is assumed to be 70%, allowing additional replicas to be provisioned during increased load.
- A Persistent Volume Claim (PVC) size of 5Gi has been considered sufficient for the expected database workload and data storage requirements.

---

3. Solution Overview

- A Bookshelf REST API microservice was developed using Node.js with PostgreSQL as the backend database. The application was containerized using Docker, and the image was published to Docker Hub for deployment on Kubernetes.
- The solution follows a two-tier architecture:

End User
|
Ingress
|
API Service (ClusterIP)
|
Bookshelf API Deployment (4 Replicas)
|
PostgreSQL Service (ClusterIP)
|
PostgreSQL StatefulSet (1 Replica)
|
Persistent Volume Claim (PVC)

- External requests are routed through Ingress to the API tier. The API communicates with the PostgreSQL database through an internal ClusterIP Service.
- Application configuration is managed using ConfigMaps, while sensitive information such as database credentials is stored using Kubernetes Secrets.
- The solution leverages Kubernetes-native resources to provide external access, persistence, configuration management, autoscaling, and self-healing capabilities while adhering to the assignment requirements and FinOps principles.

---

4. Justification for the Resources Utilized

Resource	**Purpose**
Deployment	Deploy and manage the stateless API workload with support for scaling, rolling updates, and self-healing.

Resource	Purpose
StatefulSet	Deploy and manage PostgreSQL database while ensuring persistent data storage and state management.
ClusterIP Service	Enable internal communication between the API tier and database tier within the Kubernetes cluster.
Ingress	Expose the API tier externally through a single-entry point and route incoming requests to the application.
ConfigMap	Externalize application configuration and avoid hardcoding configuration values.
Secret	Securely store and manage sensitive information such as database credentials.
Persistent Volume Claim (PVC)	Provide persistent storage for PostgreSQL and ensure data survives pod recreation.
Horizontal Pod Autoscaler (HPA)	Automatically scale API replicas based on CPU utilization to handle varying workloads efficiently.

---

5. FinOps Considerations

- CPU and memory requests and limits have been configured to prevent overprovisioning of cluster resources.
- Horizontal Pod Autoscaler (HPA) has been configured to automatically scale the API tier between 4 and 8 replicas based on CPU utilization.
- A target CPU utilization threshold of 70% has been configured to trigger scaling activities only when required.
- Persistent storage has been provisioned according to application requirements, avoiding excessive storage allocation.
- A two-node Kubernetes cluster has been utilized to provide the required capacity while maintaining cost efficiency.

These measures help balance application performance, scalability, availability, and infrastructure cost optimization.